Tugas Pertemuan 2	
IF21W0508 – Praktikum Web	
NPM	243040023
Nama	Dimas Muhammad Fadhil
Kelas	E
Link Github Repository	https://github.com/dimasbimo/PW2026_243040023/tree/main/tugas2

Bagian 1. Screenshot Semua Kode

App.jsx

```jsx
import { useEffect, useState } from 'react'
import FormTransaksi from './FormTransaksi'
import RingkasanSaldo from './RingkasanSaldo'
import DaftarTransaksi from './DaftarTransaksi'
import './App.css'

const STORAGE_KEY = 'catatan-keuangan-mini-transaksi'
const formatRupiah = (nilai) => new Intl.NumberFormat('id-ID', {
  style: 'currency', currency: 'IDR', maximumFractionDigits: 2,
}).format(nilai)

function muatTransaksi() {
  try {
    const data = JSON.parse(localStorage.getItem(STORAGE_KEY) || '[]')
    if (!Array.isArray(data)) return []
    return data.filter((item) => item && typeof item.id === 'string'
      && typeof item.keterangan === 'string' && item.keterangan.trim()
      && Number.isFinite(item.nominal) && item.nominal > 0
      && ['Pemasukan', 'Pengeluaran'].includes(item.jenis))
  } catch {
    return []
  }
}

function App() {
  // Lazy initializer membaca data sebelum efek penyimpanan pertama berjalan.
  const [transaksi, setTransaksi] = useState(muatTransaksi)

  useEffect(() => {
    try {
      localStorage.setItem(STORAGE_KEY, JSON.stringify(transaksi))
    } catch (error) {
      console.error('Transaksi gagal disimpan di browser.', error)
    }
  }, [transaksi])

  const pemasukan = transaksi.reduce((total, item) =>
    item.jenis === 'Pemasukan' ? total + item.nominal : total, 0)
  const pengeluaran = transaksi.reduce((total, item) =>
    item.jenis === 'Pengeluaran' ? total + item.nominal : total, 0)
  const saldo = pemasukan - pengeluaran

  function tambahTransaksi(data) {
    setTransaksi((sebelumnya) => [{ ...data, id: crypto.randomUUID() },
...sebelumnya])
  }

  function hapusTransaksi(id) {
    setTransaksi((sebelumnya) => sebelumnya.filter((item) => item.id !== id))
  }
```

Gambar 1. Komponen utama, pemuatan data awal, dan pengelolaan transaksi

```
  return (
    <div className="app-shell">
      <main className="dashboard">
        <div className="page-heading">
          <div>
            <h1>KAS MAHASISWA</h1>
            <p className="subtitle">Kelola iuran dan pengeluaran kas mahasiswa
dalam satu tempat.</p>
          </div>
        </div>
        <RingkasanSaldo pemasukan={pemasukan} pengeluaran={pengeluaran} saldo=
{saldo} formatRupiah={formatRupiah} />
        <div className="content-grid">
          <FormTransaksi onTambah={tambahTransaksi} />
          <DaftarTransaksi transaksi={transaksi} onHapus={hapusTransaksi}
formatRupiah={formatRupiah} />
        </div>
      </main>
    </div>
  )
}

export default App
```

Gambar 2. Komponen utama, pemuatan data awal, dan pengelolaan transaksi

DaftarTransaksi.jsx

```
export default function DaftarTransaksi({ transaksi, onHapus, formatRupiah })
{
  return (
    <section className="panel history-panel" aria-labelledby="history-title">
      <div className="history-heading">
        <div><h2 id="history-title">Riwayat kas</h2><p>Pantau iuran masuk dan
penggunaan dana mahasiswa.</p></div>
        <span className="transaction-count" aria-live="polite">
{transaksi.length} transaksi</span>
      </div>
      {transaksi.length === 0 ? (
        <div className="empty-state">
          <div className="empty-icon" aria-hidden="true">
            <svg width="38" height="38" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round"><path d="M20 8V6a2 2 0 0 0-2-2H6a3 3 0 0 0 0
6h14v10H6a3 3 0 0 1-3-3V7" /><path d="M20 12h-4a2 2 0 0 0 0 4h4" /><path
d="M17 14h.01M6 7h11" /></svg>
          </div>
          <h3>Belum ada transaksi kas</h3>
          <p>Mulai dengan mencatat iuran mahasiswa atau pengeluaran kegiatan
melalui formulir.</p>
        </div>
      ) : (
        <ul className="transaction-list">
          {transaksi.map((item) => {
            const masuk = item.jenis === 'Pemasukan'
            return (
              <li className="transaction-item" key={item.id}>
                <span className={masuk ? 'transaction-icon income-icon' :
```

Gambar 3. Tampilan riwayat, kondisi kosong, dan tombol hapus transaksi.

```
'transaction-icon expense-icon'} aria-hidden="true">{masuk ? '↙' : '↗'}</span>
                <div className="transaction-detail"><h3>{item.keterangan}</h3>
<span>{masuk ? 'Kas masuk' : 'Kas keluar'}</span></div>
                <p className={masuk ? 'transaction-amount income' :
'transaction-amount expense'}>{masuk ? '+' : '−'}{formatRupiah(item.nominal)}
</p>
                <button className="delete-button" type="button" onClick={() =>
onHapus(item.id)} aria-label={'Hapus transaksi kas ' + item.keterangan}
title="Hapus transaksi kas">
                  <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="1.7" strokeLinecap="round"
strokeLinejoin="round" aria-hidden="true"><path d="M3 6h18M9 6V4h6v2M5 6l1
14h12l1-14M10 10v6M14 10v6" /></svg>
                </button>
              </li>
            )
          })}
        </ul>
      )}
    </section>
  )
}
```

Gambar 4. Tampilan riwayat, kondisi kosong, dan tombol hapus transaksi.

FormTransaksi.jsx

```
import { useState } from 'react'

export default function FormTransaksi({ onTambah }) {
  const [keterangan, setKeterangan] = useState('')
  const [nominal, setNominal] = useState('')
  const [jenis, setJenis] = useState('Pemasukan')
  const [pesanError, setPesanError] = useState('')

  function handleSubmit(event) {
    event.preventDefault()
    // Nilai input berupa teks, jadi ubah menjadi angka sebelum dihitung.
    const nilaiNominal = Number(nominal)
    if (!keterangan.trim() || !nominal.trim() ||
!Number.isFinite(nilaiNominal) || nilaiNominal <= 0) {
      setPesanError('Isi keterangan dan nominal yang lebih besar dari 0.')
      return
    }
    onTambah({ keterangan: keterangan.trim(), nominal: nilaiNominal, jenis })
    setKeterangan('')
    setNominal('')
    setPesanError('')
  }

  return (
    <section className="panel form-panel" aria-labelledby="form-title">
      <div className="section-heading">
        <span className="section-icon" aria-hidden="true">+</span>
        <div><h2 id="form-title">Catat transaksi kas</h2><p>Dari iuran hingga
dana kegiatan mahasiswa.</p></div>
      </div>
      <form onSubmit={handleSubmit}>
        <div className="field">
          <label htmlFor="keterangan">Keterangan</label>
```

Gambar 5. State formulir, validasi, dan awal susunan input transaksi.

```
          <input id="keterangan" type="text" placeholder="Contoh: Iuran kas
mingguan" value={keterangan} onChange={(event) =>
setKeterangan(event.target.value)} maxLength={100} required />
        </div>
        <div className="field">
          <label htmlFor="nominal">Nominal</label>
          <div className="money-input"><span aria-hidden="true">Rp</span>
<input id="nominal" type="number" inputMode="decimal" min="0.01" step="0.01"
placeholder="0" value={nominal} onChange={(event) =>
setNominal(event.target.value)} required aria-describedby="nominal-hint" />
</div>
          <small id="nominal-hint">Masukkan nominal tanpa pemisah ribuan.
</small>
        </div>
        <div className="field">
          <label htmlFor="jenis">Jenis transaksi</label>
          <select id="jenis" value={jenis} onChange={(event) =>
setJenis(event.target.value)}>
            <option value="Pemasukan">Kas masuk</option>
            <option value="Pengeluaran">Kas keluar</option>
          </select>
        </div>
        {pesanError && <p className="form-error" role="alert">{pesanError}
</p>}
        <button className="submit-button" type="submit"><span aria-
hidden="true">+</span> Simpan transaksi</button>
      </form>
    </section>
  )
}
```

Gambar 6. State formulir, validasi, dan awal susunan input transaksi.

RingkasanSaldo.jsx

```
export default function RingkasanSaldo({ pemasukan, pengeluaran, saldo,
formatRupiah }) {
  return (
    <section className="summary-grid" aria-label="Ringkasan kas mahasiswa"
aria-live="polite" aria-atomic="true">
      <article className="summary-card balance-card">
        <div className="card-top"><p>Saldo kas</p><span className="card-
symbol" aria-hidden="true">◆</span></div>
        <p className="summary-value">{formatRupiah(saldo)}</p>
        <p className="card-caption">Kas masuk dikurangi kas keluar</p>
      </article>
      <article className="summary-card">
        <div className="card-top"><p>Total kas masuk</p><span className="card-
symbol income-icon" aria-hidden="true">↙</span></div>
        <p className="summary-value income">{formatRupiah(pemasukan)}</p>
        <p className="card-caption">Iuran dan dana kegiatan yang diterima</p>
      </article>
      <article className="summary-card">
        <div className="card-top"><p>Total kas keluar</p><span
className="card-symbol expense-icon" aria-hidden="true">↗</span></div>
        <p className="summary-value expense">{formatRupiah(pengeluaran)}</p>
        <p className="card-caption">Pengeluaran untuk kegiatan mahasiswa</p>
      </article>
    </section>
  )
}
```

Gambar 7. Komponen penampil saldo kas, total kas masuk, dan total kas keluar.

Bagian 2. Penjelasan Kode

1. App.jsx

Komponen App menjadi pusat pengelolaan data aplikasi Kas Mahasiswa. Berkas ini mengimpor FormTransaksi, RingkasanSaldo, DaftarTransaksi, dan App.css. Daftar transaksi dikelola menggunakan *useState(muatTransaksi)*. Fungsi pemuat diberikan sebagai lazy initializer sehingga data awal dibaca sebelum efek penyimpanan pertama dijalankan.

Fungsi muatTransaksi membaca data dari localStorage dengan kunci *catatan-keuangan-mini-transaksi*. JSON diproses di dalam try/catch. Data harus berupa array, kemudian setiap item disaring agar memiliki id berupa string, keterangan yang tidak kosong setelah trim, nominal berupa angka finite yang lebih besar dari nol, serta jenis Pemasukan atau Pengeluaran. Data yang tidak sesuai tidak digunakan. Jika pembacaan JSON gagal, fungsi mengembalikan array kosong.

useEffect menyimpan perubahan daftar transaksi kembali ke localStorage. Total pemasukan dan pengeluaran dihitung memakai *reduce*, sedangkan saldo merupakan selisih keduanya. *Intl.NumberFormat* mengatur format uang dengan lokal id-ID, mata uang IDR, dan maksimal dua angka desimal.

Fungsi *tambah Transaksi* membuat identitas unik melalui *crypto.randomUUID()* dan menempatkan transaksi terbaru pada awal array. Fungsi *hapusTransaksi* memakai filter untuk mengeluarkan item dengan id yang dipilih. Nilai ringkasan, daftar transaksi, serta fungsi tambah dan hapus diteruskan ke komponen anak melalui props.

2. DaftarTransaksi.jsx

Komponen ini menerima props transaksi, onHapus, dan formatRupiah. Jumlah transaksi ditampilkan pada bagian judul. Jika array kosong, komponen menampilkan ilustrasi SVG, judul kondisi kosong, dan petunjuk pencatatan. Jika data tersedia, map menghasilkan elemen daftar dengan *key={item.id}*.

Variabel masuk membedakan pemasukan dan pengeluaran. Nilainya menentukan ikon, warna, label Kas masuk atau Kas keluar, serta tanda tambah atau kurang di depan nominal. Setiap item mempunyai tombol hapus yang memanggil *onHapus(item.id)*. Atribut aria-label menyertakan keterangan transaksi agar tujuan tombol lebih jelas bagi pembaca layar.

3. FormTransaksi.jsx

Komponen FormTransaksi menyediakan input keterangan, nominal, dan jenis transaksi. State keterangan dan nominal diawali dengan teks kosong, jenis diawali dengan Pemasukan, sedangkan pesanError menyimpan pesan validasi. Setiap input merupakan controlled input karena nilainya mengikuti state dan diperbarui melalui onChange.

Saat formulir dikirim, *event.preventDefault()* mencegah pemuatan ulang halaman. Nilai nominal dikonversi menggunakan *Number(nominal)* agar dapat dihitung sebagai angka. Pemeriksaan menolak keterangan yang hanya berisi spasi, nominal kosong, nilai nonfinite, serta nominal yang kurang dari atau sama dengan nol.

Data yang valid diteruskan ke onTambah dengan keterangan yang sudah di-trim, nominal berbentuk angka, dan jenis yang dipilih. Setelah pencatatan berhasil, keterangan, nominal, dan pesan kesalahan dikosongkan. Pilihan jenis transaksi tetap dipertahankan sehingga pengguna dapat mencatat beberapa transaksi dengan jenis yang sama.

Atribut HTML melengkapi validasi JavaScript. Input keterangan memakai required dan maxLength 100 karakter. Input nominal memakai tipe number, inputMode decimal, serta min dan step 0.01. Label dihubungkan ke input melalui htmlFor dan id. Pesan kesalahan ditampilkan dengan role alert ketika tersedia.

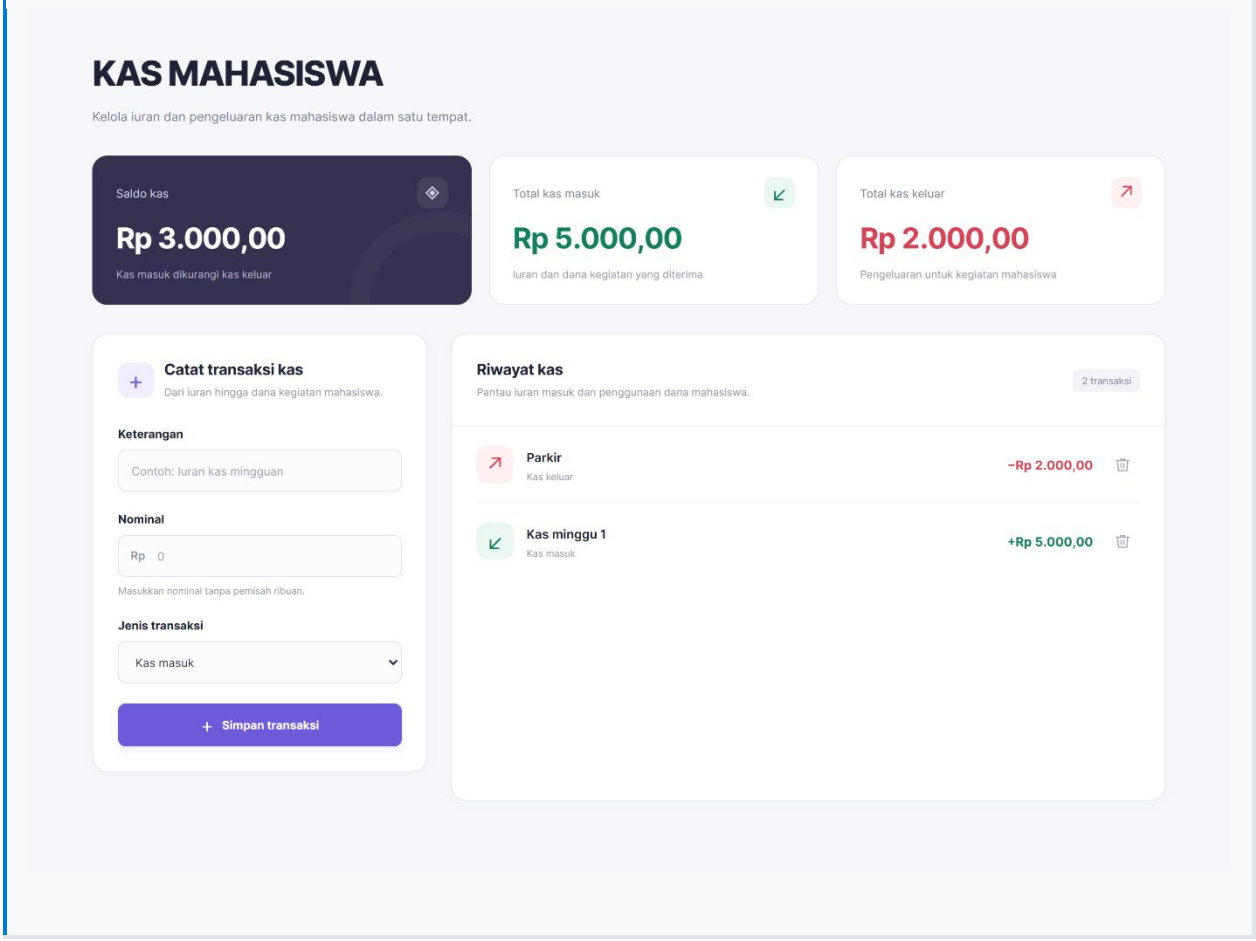
4. RingkasanSaldo.jsx

Komponen RingkasanSaldo menerima pemasukan, pengeluaran, saldo, dan formatRupiah melalui props. Komponen ini tidak menyimpan state transaksi sendiri. Tiga kartu menampilkan saldo kas, total kas masuk, dan total kas keluar dengan format uang yang konsisten.

Setiap penambahan atau penghapusan transaksi mengubah state pada App, memperbarui hasil perhitungan, dan menampilkan nilai terbaru melalui props. Warna hijau menunjukkan kas masuk dan warna merah menunjukkan kas keluar, sedangkan saldo memakai kartu berwarna gelap. Atribut aria-live polite dan aria-atomic mendukung pemberitahuan perubahan ringkasan kepada pengguna pembaca layar. Ikon dekoratif diberi aria-hidden.

Pemisahan keempat komponen membuat pengelolaan data, formulir, ringkasan, dan riwayat lebih mudah dibaca serta dipelihara.

Bagian 3. Screenshot Hasil Antarmuka



Bagian 4. Penjelasan Hasil Antarmuka

1. Judul dan ringkasan kas

Halaman menampilkan judul Kas Mahasiswa dan subjudul yang menjelaskan fungsi pencatatan iuran serta pengeluaran kas. Tiga kartu ringkasan memperlihatkan saldo, total kas masuk, dan total kas keluar. Pada contoh pencatatan, kas masuk Rp5.000,00 dan kas keluar Rp2.000,00 menghasilkan saldo Rp3.000,00.

2. Formulir dan riwayat transaksi

Formulir menyediakan keterangan, nominal, pilihan jenis, dan tombol Simpan transaksi. Transaksi baru muncul pada urutan paling atas riwayat. Setiap item menampilkan keterangan, jenis, nominal, dan tombol hapus. Ikon serta warna hijau dan merah membantu membedakan kas masuk dan kas keluar. Setelah data dihapus, ringkasan mengikuti perubahan daftar transaksi.

3. Penyesuaian tema dan penyederhanaan tampilan

Tema Catatan Keuangan Mini diubah menjadi Kas Mahasiswa. Istilah pada ringkasan, pilihan jenis transaksi, riwayat, dan contoh keterangan disesuaikan dengan iuran serta kegiatan mahasiswa. Header tambahan, badge penyimpanan, footer, titik dekoratif pada judul, slogan, catatan bawah formulir, dan caption kondisi kosong dihapus. Isi halaman difokuskan pada ringkasan, pencatatan, dan riwayat.

CSS untuk header, merek, badge penyimpanan, dekorasi judul, dan footer telah dibersihkan. Aturan kontrol formulir yang berulang pada index.css juga digabungkan. Pada sumber yang diperiksa, selector *.eyebrow, .form-note*, dan *.empty-caption* masih tersedia walaupun elemen terkait sudah dihapus. Selector tersebut dapat dibersihkan pada pemeliharaan berikutnya.

4. Tata letak dan penyimpanan

Tata letak memakai Grid dan Flexbox. Pada desktop, formulir dan riwayat disusun berdampingan. Media query pada 900 dan 680 piksel menyesuaikan ukuran panel dan susunan konten; pada layar kecil formulir dan riwayat menjadi satu kolom. Pengaturan 1400 piksel menyesuaikan jarak atas dashboard. Data disimpan secara lokal pada browser melalui localStorage sehingga tidak otomatis tersinkron ke perangkat atau browser lain.